NORMALIZATION

Table of Contents

1. Data Redundancy

2. Types of Anomalies

3. What is Normalization?

4. Normalization Forms — Overview

5. First Normal Form (1NF)

6. Second Normal Form (2NF)

7. Third Normal Form (3NF)

8. Boyce-Codd Normal Form (BCNF)

9. Fourth Normal Form (4NF)

10. Fifth Normal Form (5NF)

1. Data Redundancy

- Data redundancy means having the same data stored at multiple places within a database.
- Data redundancy leads to anomalies (errors) in the database.
- It becomes very hard to maintain a database that has redundant data.

Example — Data Redundancy:

NAME	ROLL NO	SECTION
SAI	21	A
SAI	21	A
SHIVA	22	B

Notice that the row for "SAI, 21, A" is repeated — this duplication is an example of data redundancy.

2. Types of Anomalies

Data redundancy causes anomalies. The three types of anomalies are:

- Insertion Anomalies — difficulty inserting data due to missing/dependent information.
- Updation Anomalies — the need to update the same data in multiple places, risking inconsistency if missed.
- Deletion Anomalies — deleting a record unintentionally removes other, unrelated information stored in the same row.

3. What is Normalization?

Normalization is the process of organizing data in a database to reduce data redundancy and improve data integrity.

It involves dividing large tables into smaller ones and defining relationships between them to ensure data consistency.

The main goals of normalization are:

- Eliminate data redundancy (duplicate data).
- Ensure data dependencies are logical.
- Improve data integrity (missing values) and consistency (correctness).

4. Normalization Forms — Overview

Normalization is typically carried out through a series of normal forms (NF), each addressing specific issues:

- First Normal Form (1NF) — Eliminate repeating groups.
- Second Normal Form (2NF) — Eliminate partial dependency.
- Third Normal Form (3NF) — Eliminate transitive dependency.
- Boyce-Codd Normal Form (BCNF) — A stricter version of 3NF.
- Fourth Normal Form (4NF) — Handle multi-valued dependencies.
- Fifth Normal Form (5NF) — Handle join dependencies.

5. First Normal Form (1NF)

- A relation should be in 1NF if it contains only atomic values.
- It states that an attribute of a table cannot hold multiple values.

- 1NF does not allow multi-valued attributes.

Before applying 1NF (multi-valued attribute LANGUAGES_KNOWN):

ROLL NO	NAME	BRANCH	LANGUAGES_KNOWN
101	SAI	CSE	TELUGU, ENGLISH
102	SHIVA	AIML	ENGLISH, HINDI
103	RAJU	DS	ENGLISH, HINDI

After applying 1NF (each value made atomic — one language per row):

ROLL NO	NAME	COURSE	LANGUAGES_KNOWN
101	SAI	CSE	TELUGU
101	SAI	CSE	ENGLISH
102	SHIVA	AIML	ENGLISH
102	SHIVA	AIML	HINDI
103	RAJU	DS	ENGLISH
103	RAJU	DS	HINDI

6. Second Normal Form (2NF)

- The table must already be in 1NF.
- There are no partial dependencies — i.e., no non-prime attribute depends on only part of a composite primary key.
- The primary key should not contain duplicate columns.

Table before applying 2NF (roll no repeats due to multiple languages, a partial dependency on part of the key):

ROLL NO [PK]	NAME	COURSE	LANGUAGES_KNOWN

101	SAI	CSE	TELUGU
101	SAI	CSE	ENGLISH
102	SHIVA	AIML	ENGLISH
102	SHIVA	AIML	HINDI
103	RAJU	DS	ENGLISH
103	RAJU	DS	HINDI

After applying 2NF — split into two related tables:

ROLL NO	NAME	BRANCH
101	SAI	CSE
102	SHIVA	AIML
103	RAJU	DS

ROLL NO	LANGUAGES_KNOWN
101	TELUGU
101	ENGLISH
102	ENGLISH
102	HINDI
103	ENGLISH
103	HINDI

7. Third Normal Form (3NF)

- The table must already be in 2NF.

- There are no transitive dependencies — i.e., no non-prime attribute (N.P.A) depends on another non-prime attribute.

Table before applying 3NF (FEE depends on BRANCH, not directly on ROLL NO — a transitive dependency):

ROLL NO [PK]	NAME	BRANCH [N.P.A]	FEE [N.P.A]
101	SAI	CSE	60000
102	SHIVA	AIML	75000
103	RAMESH	DS	55000
104	SURESH	ECE	30000

After applying 3NF — split into two related tables:

ROLL NO [PK]	NAME [N.P.A]	BRANCH [N.P.A]
101	SAI	CSE
102	SHIVA	AIML
103	SURESH	DS
104	RAMESH	ECE

BRANCH [PK]	FEE [N.P.A]
CSE	60000
AIML	75000
DS	55000
ECE	35000

8. Boyce-Codd Normal Form (BCNF)

- The table must already be in 3NF.
- Every determinant must be a candidate key — i.e., no non-trivial functional dependency should exist where the determinant is not a superkey.
- In other words: if A depends on B, then B should be a super key.

Table before applying BCNF:

ROLL NO [PK]	NAME	BRANCH_ID	BRANCH NAME
101	SAI	121	CSE
102	SIVA	122	AIML
103	RAMESH	123	DS
104	SURESH	121	CSE

After applying BCNF — split into two related tables:

ROLL NO [PK]	NAME
101	SAI
102	SHIVA
103	RAMESH
104	SURESH

BRANCH_ID	BRANCH_NAME
121	CSE
122	AIML
123	DS

9. Fourth Normal Form (4NF)

- The table must already be in BCNF.
- It has no multi-valued dependencies — i.e., attributes are independent of each other and depend only on the primary key.

Table before applying 4NF (COURSE and HOBBY are independent multi-valued facts about the same ROLL NO):

ROLL NO [PK]	COURSE [N.P.A]	HOBBY [N.P.A]
101	CSE	DANCING
102	AIML	SLEEPING
103	DS	SINGING

After applying 4NF — split into two independent tables:

ROLL NO [PK]	COURSE [N.P.A]
101	CSE
102	AIML
103	DS

ROLL NO [PK]	HOBBY [N.P.A]
101	DANCING
102	SLEEPING
103	SINGING

10. Fifth Normal Form (5NF)

- The table must already be in 4NF.
- It does not consist of any join dependencies.

- 5NF is satisfied when tables present in the DBMS are broken down into multiple tables to ensure there is no redundancy when they are re-joined.

Table before applying 5NF:

SUBJECT	FACULTY	YEAR
C	SAI	1
C	SHIVA	1
JAVA	RAJU	2
DBMS	RAMESH	3

After applying 5NF — split into three tables to remove the join dependency:

SUBJECT	FACULTY
C	SAI
C	SHIVA
JAVA	RAJU
DBMS	RAMESH

FACULTY	YEAR
SAI	1
SHIVA	1
RAJU	2
RAMESH	3

SUBJECT	YEAR

C	1
JAVA	2
DBMS	3

Normalization Practical — 1NF → 2NF → 3NF

Step 0: Unnormalized Table

First create a table containing multiple values in the same column

```
CREATE TABLE student_details (
   student_id INT,
   student_name VARCHAR(50),
   courses VARCHAR(100),
   trainers VARCHAR(100),
   trainer_phones VARCHAR(100)
);

INSERT INTO student_details
VALUES
(101, 'Ravi', 'Java, Python', 'Kumar, Ramesh', '9876543210, 9876543211'),
(102, 'Priya', 'Python, SQL', 'Ramesh, Suresh', '9876543211, 9876543212'),
(103, 'Arun', 'Java, SQL', 'Kumar, Suresh', '9876543210, 9876543212');

SELECT * FROM student_details;
```

Output:

student_id	student_name	courses	trainers	trainer_phones
101	Ravi	Java, Python	Kumar, Ramesh	9876543210, 9876543211

102	Priya	Python, SQL	Ramesh, Suresh	9876543211, 9876543212
103	Arun	Java, SQL	Kumar, Suresh	9876543210, 9876543212

Problem

courses, trainers, and trainer_phones contain **multiple values**.

So this table is **not in 1NF**.

Step 1: Convert to 1NF

1NF Rule

Every column should contain a single/atomic value.

Create a new table:

```
CREATE TABLE student_course_1nf (
   student_id INT,
   student_name VARCHAR(50),
   course VARCHAR(30),
   trainer VARCHAR(50),
   trainer_phone VARCHAR(15)
);


INSERT INTO student_course_1nf
VALUES
(101, 'Ravi', 'Java', 'Kumar', '9876543210'),
(101, 'Ravi', 'Python', 'Ramesh', '9876543211'),
(102, 'Priya', 'Python', 'Ramesh', '9876543211'),
(102, 'Priya', 'SQL', 'Suresh', '9876543212'),
(103, 'Arun', 'Java', 'Kumar', '9876543210'),
(103, 'Arun', 'SQL', 'Suresh', '9876543212');
```

```
SELECT * FROM student_course_1nf;
```

Output:

student_id	student_name	course	trainer	trainer_phone
101	Ravi	Java	Kumar	9876543210
101	Ravi	Python	Ramesh	9876543211
102	Priya	Python	Ramesh	9876543211
102	Priya	SQL	Suresh	9876543212
103	Arun	Java	Kumar	9876543210
103	Arun	SQL	Suresh	9876543212

Now every cell contains a **single value**.

✅ **Table is in 1NF.**

Step 2: Convert 1NF to 2NF

Now we identify the **primary key**.

A student can attend multiple courses.

```
ALTER TABLE student_course_1nf ADD PRIMARY KEY (student_id, course);


CREATE TABLE student_2nf (
   student_id INT PRIMARY KEY,
   student_name VARCHAR(50)
);


INSERT INTO student_2nf
```

```
VALUES
(101, 'Ravi'),
(102, 'Priya'),
(103, 'Arun');

CREATE TABLE enrollment_2nf (
   student_id INT,
   course VARCHAR(30),
   trainer VARCHAR(50),
   trainer_phone VARCHAR(15),
   PRIMARY KEY (student_id, course),
   FOREIGN KEY (student_id) REFERENCES student_2nf(student_id)
);

INSERT INTO enrollment_2nf
VALUES
(101, 'Java', 'Kumar', '9876543210'),
(101, 'Python', 'Ramesh', '9876543211'),
(102, 'Python', 'Ramesh', '9876543211'),
(102, 'SQL', 'Suresh', '9876543212'),
(103, 'Java', 'Kumar', '9876543210'),
(103, 'SQL', 'Suresh', '9876543212');
```

Now we have:

STUDENT

student_id	student_name
101	Ravi

102	Priya
103	Arun

ENROLLMENT

student_id	course	trainer	trainer_phone
101	Java	Kumar	9876543210
101	Python	Ramesh	9876543211
102	Python	Ramesh	9876543211
102	SQL	Suresh	9876543212
103	Java	Kumar	9876543210
103	SQL	Suresh	9876543212

The student_name partial dependency has been removed.

✅ **Tables are now in 2NF.**

Step 3: Convert 2NF to 3NF

Now look at the enrollment_2nf table:

student_id

course

trainer

trainer_phone

We can identify another dependency:

trainer → trainer_phone

For example:

Kumar → 9876543210

Ramesh → 9876543211

Suresh → 9876543212

But:

student_id + course → trainer

trainer → trainer_phone

Therefore:

student_id + course → trainer → trainer_phone

This is a **transitive dependency**.

To achieve **3NF**, we remove this dependency.

Create TRAINER table

```
CREATE TABLE trainer_3nf (
   trainer_id INT PRIMARY KEY,
   trainer_name VARCHAR(50),
   trainer_phone VARCHAR(15)
);
```

Insert:

```
INSERT INTO trainer_3nf
VALUES
(1, 'Kumar', '9876543210'),
(2, 'Ramesh', '9876543211'),
(3, 'Suresh', '9876543212');
```

Create COURSE table

```
CREATE TABLE course_3nf (
   course_id INT PRIMARY KEY,
   course_name VARCHAR(30),
```

```
    trainer_id INT,
    FOREIGN KEY (trainer_id) REFERENCES trainer_3nf(trainer_id)
);
```

Insert:

```
INSERT INTO course_3nf
VALUES
(1, 'Java', 1),
(2, 'Python', 2),
(3, 'SQL', 3);
```

Create ENROLLMENT table

Now instead of storing trainer details here, we store IDs.

```
CREATE TABLE enrollment_3nf (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES student_2nf(student_id),
    FOREIGN KEY (course_id) REFERENCES course_3nf(course_id)
);
```

Insert:

```
INSERT INTO enrollment_3nf
VALUES
(101, 1),
(101, 2),
(102, 2),
(102, 3),
(103, 1),
(103, 3);
```

Final 3NF Structure

Now we have **four tables**:

STUDENT

STUDENT

---

student_id PK

student_name

COURSE

COURSE

---

course_id PK

course_name

trainer_id FK

TRAINER

TRAINER

---

trainer_id PK

trainer_name

trainer_phone

ENROLLMENT

ENROLLMENT

---

student_id PK, FK

course_id PK, FK

The relationship is:

STUDENT

|

| student_id

|

ENROLLMENT

|

| course_id

|

COURSE

|

| trainer_id

|

TRAINER

ENTITY-RELATIONSHIP (ER) MODEL

Table of Contents

1. The Entity-Relationship (ER) Model

2. Entity

3. Attributes

4. Entity and Attributes — Combined Example

5. Relationships

6. One-to-One (1:1) Relationship

7. One-to-Many (1:M) Relationship

8. Many-to-Many (M:N) Relationship

9. Cardinality

10. Practice Question

1. The Entity-Relationship (ER) Model

The Entity-Relationship (ER) Model is a conceptual framework used to describe data relationships in a structured way.

Components of the ER Model:

- Entities
- Attributes
- Relationships
- Cardinality

2. Entity

An entity represents a real-world object or concept that has a unique existence.

Entities are classified into:

- Strong Entities — exist independently (e.g., Customer, Student, Book).
- Weak Entities — depend on other entities for existence (e.g., Order_Item depends on Order).

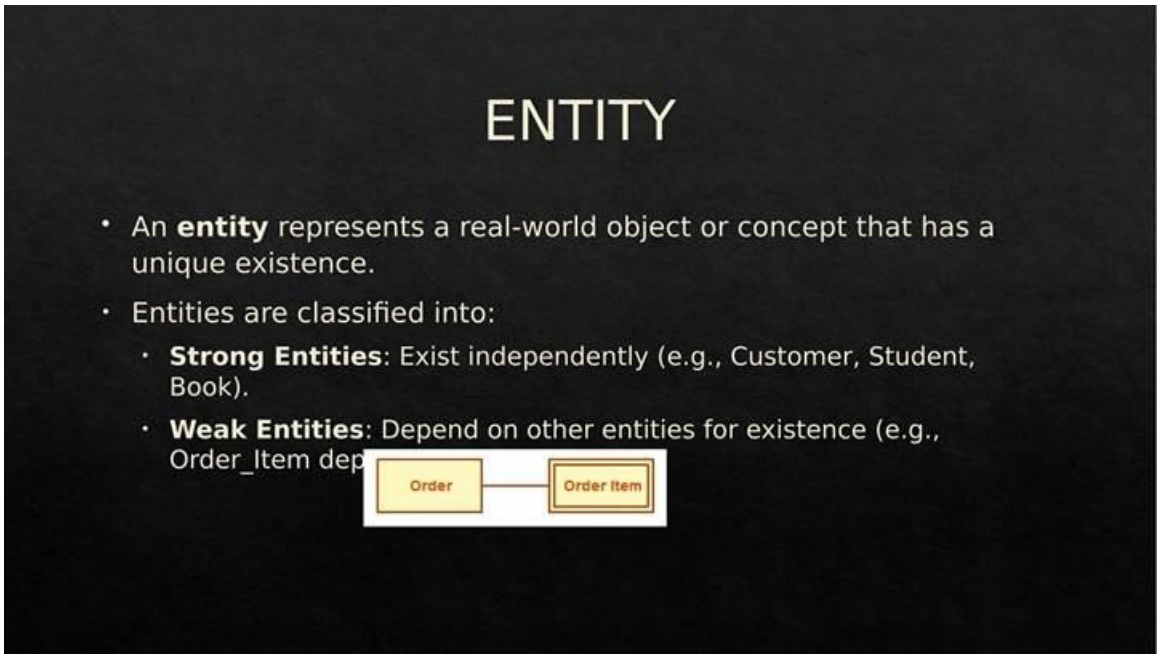


Diagram: Strong Entity vs. Weak Entity (Slide 3)

3. Attributes

Attributes describe properties or characteristics of an entity.

Types of Attributes:

- Simple (Atomic) Attributes — cannot be divided further (e.g., Name, Age).

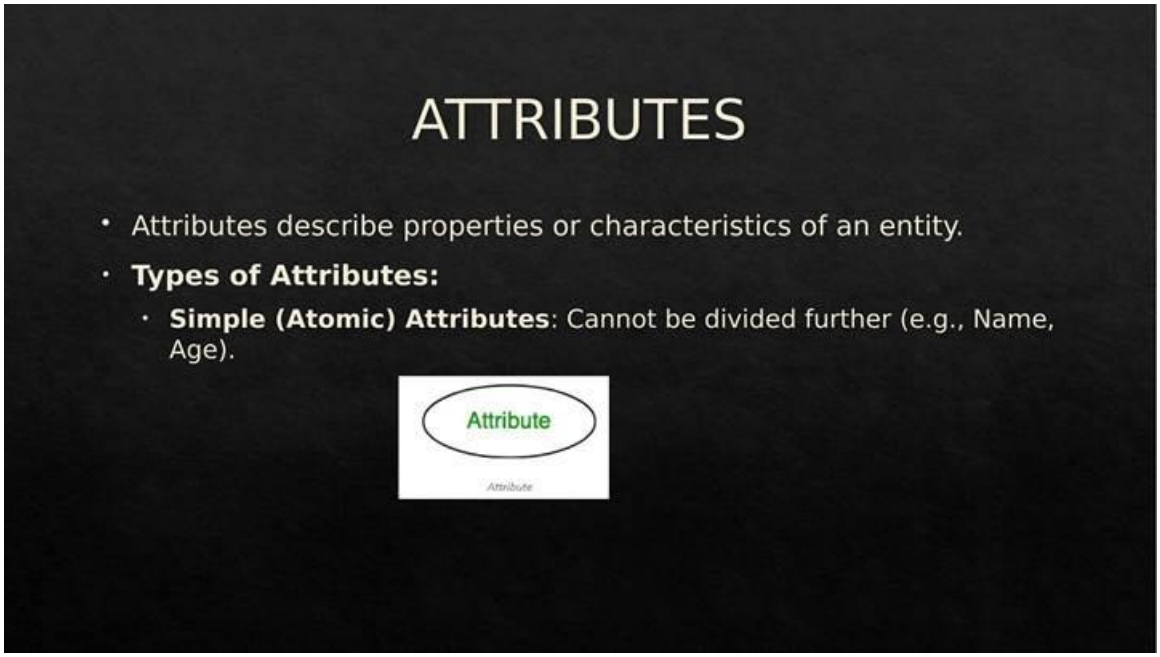


Diagram: Simple (Atomic) Attribute (Slide 4)

- Composite Attribute — can be broken down into sub-attributes (e.g., Address → Street, City, Zip).

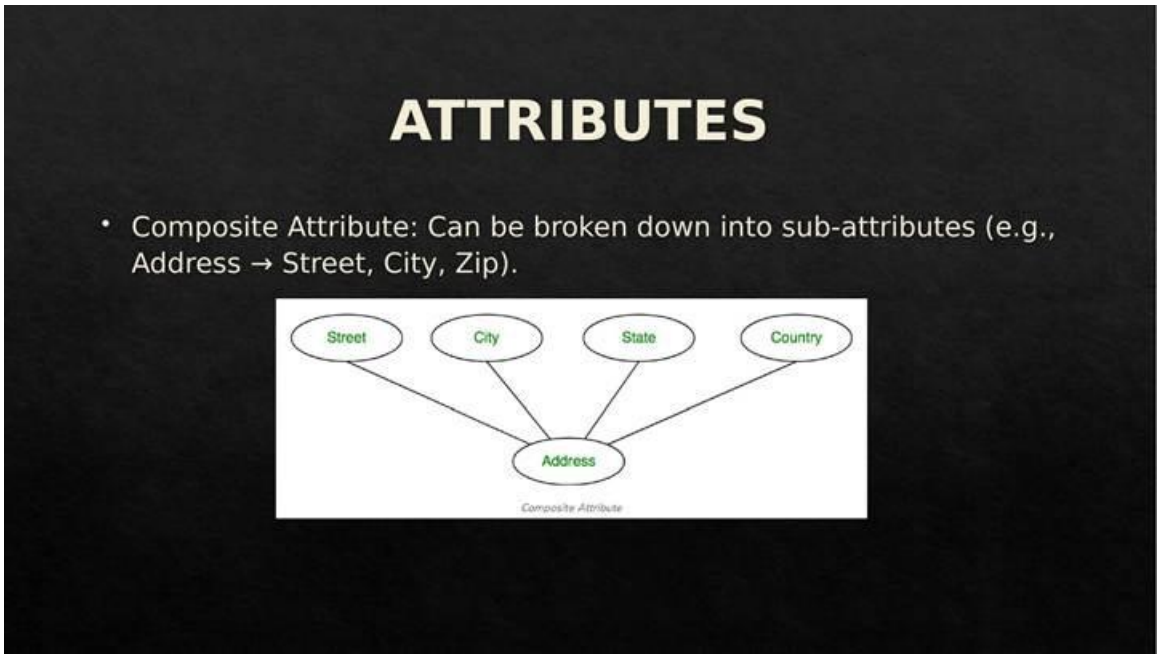


Diagram: Composite Attribute (Slide 5)

- Key Attribute — uniquely identifies an entity (Primary Key).
- Multivalued Attributes — can have multiple values (e.g., Phone Numbers).

- Derived Attributes — calculated from other attributes (e.g., Age from Date of Birth).

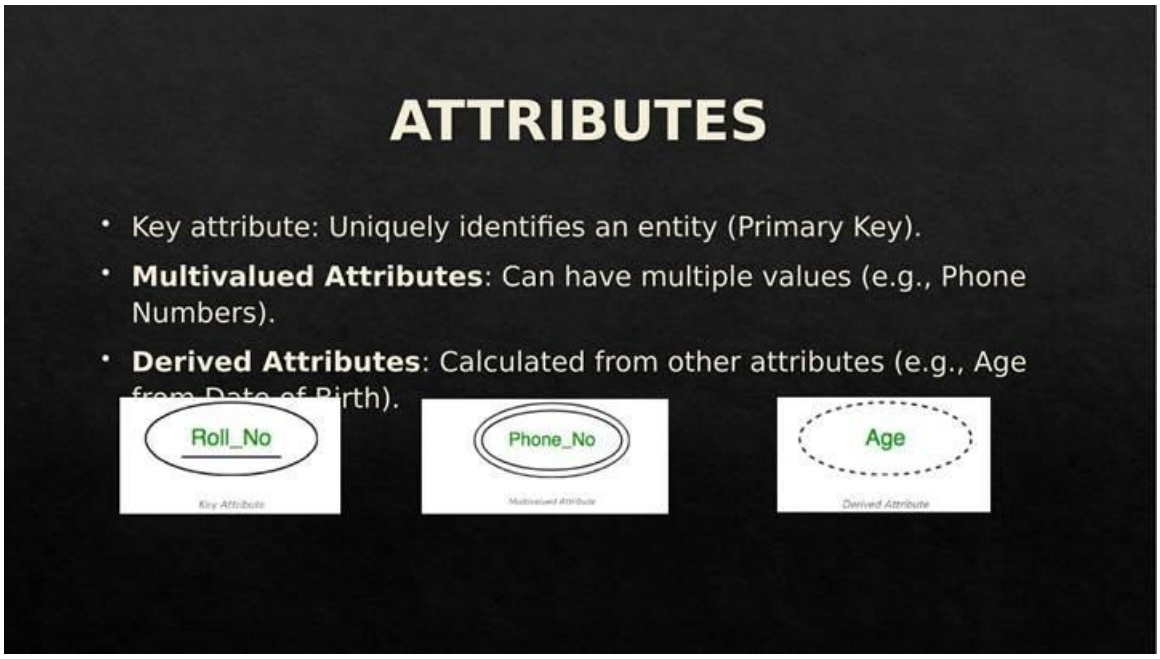


Diagram: Key, Multivalued, and Derived Attributes (Slide 6)

4. Entity and Attributes — Combined Example

The diagram below brings together an entity with its full set of attribute types in a single ER diagram.

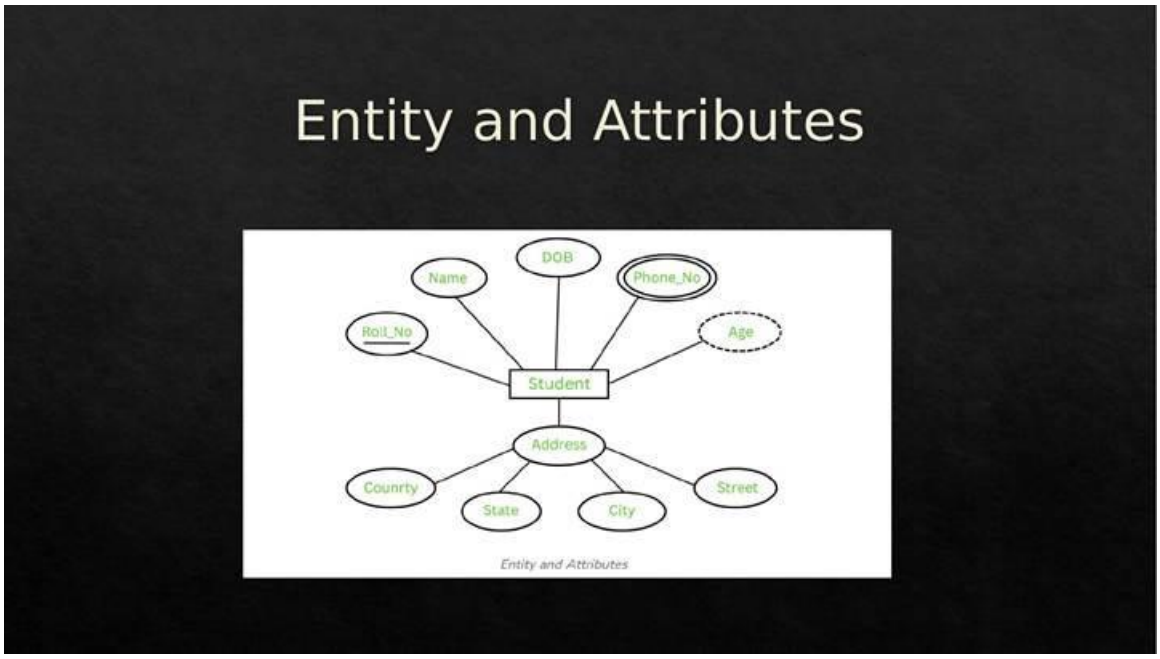


Diagram: Entity with Attributes (Slide 7)

5. Relationships

A relationship represents an association between two or more entities.

Types of Relationships:

- One-to-One (1:1)
- One-to-Many (1:M)
- Many-to-Many (M:N)

6. One-to-One (1:1) Relationship

One-to-One (1:1) → Each entity in A is related to one entity in B.

Example: COUNTRY – PRESIDENT.

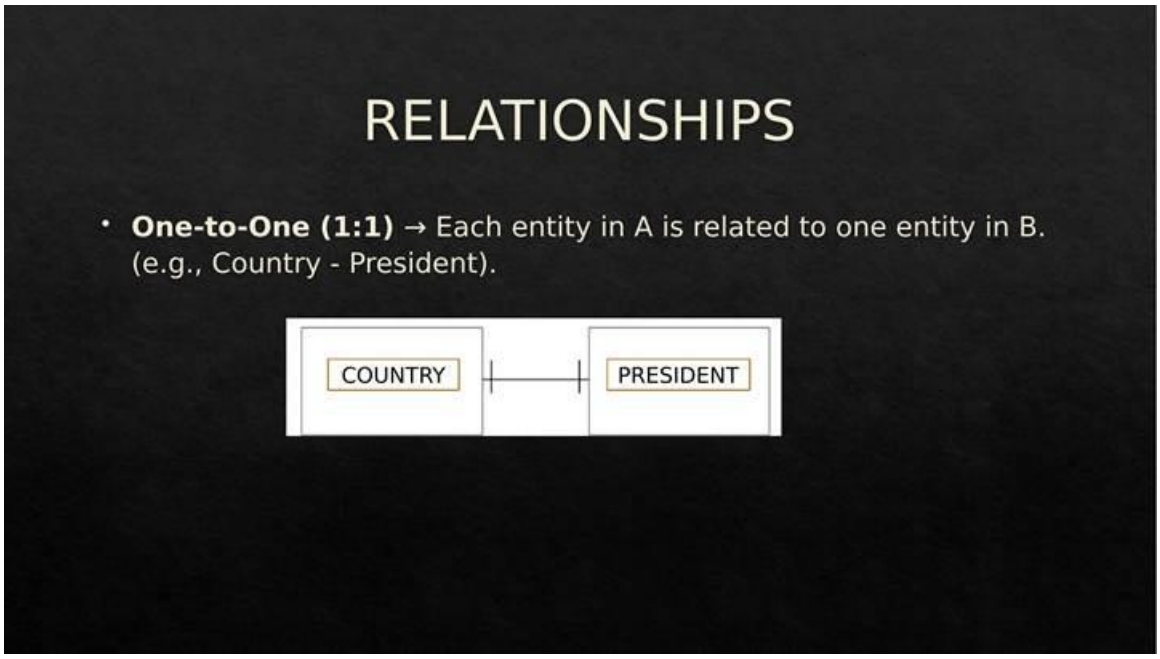


Diagram: One-to-One Relationship — Country and President (Slide 9)

7. One-to-Many (1:M) Relationship

One-to-Many (1:M) → One entity in A can relate to multiple entities in B.

Example: TEACHER – STUDENTS.

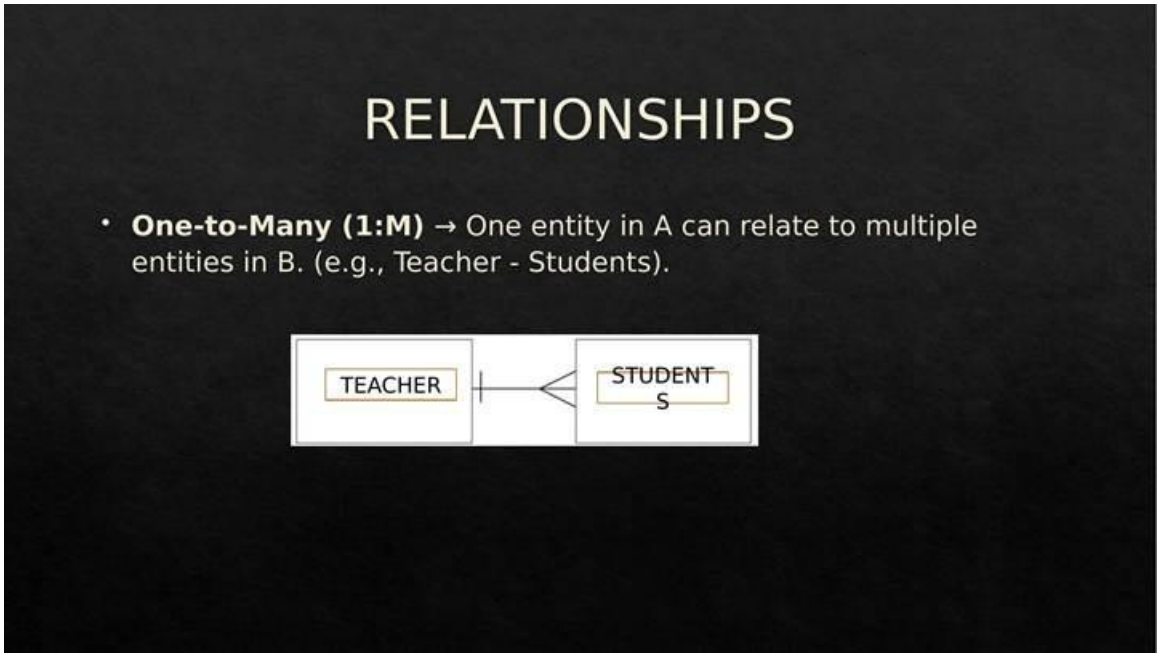


Diagram: One-to-Many Relationship — Teacher and Students (Slide 10)

8. Many-to-Many (M:N) Relationship

Many-to-Many (M:N) → Multiple entities in A relate to multiple entities in B.

Example: STUDENTS – COURSES.

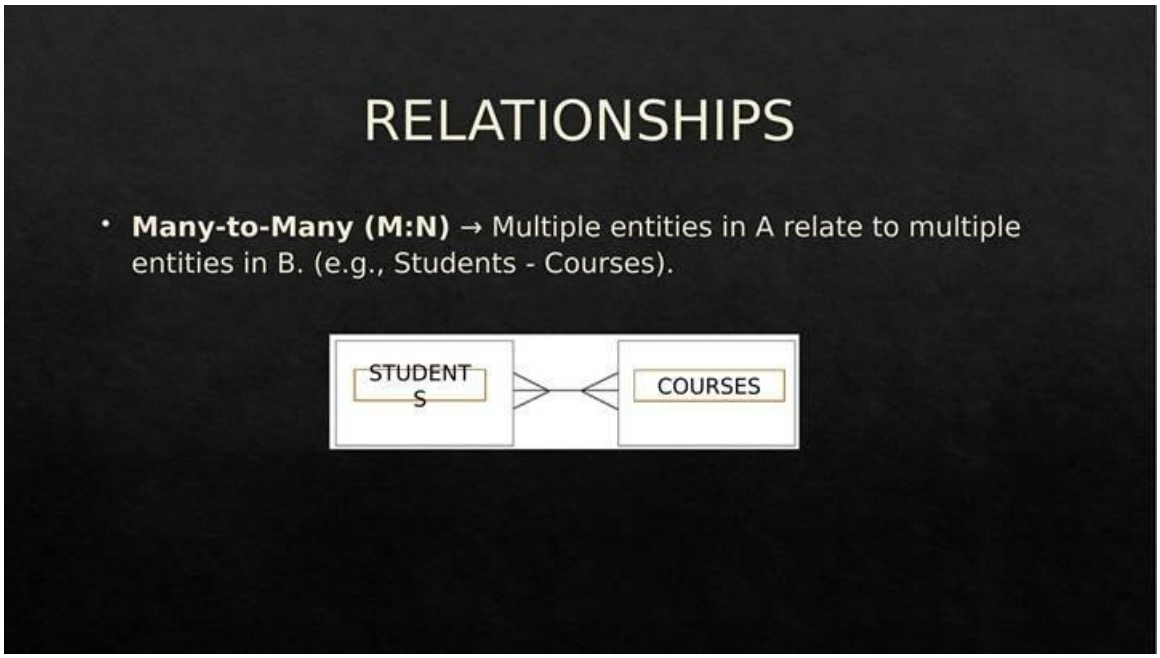


Diagram: Many-to-Many Relationship — Students and Courses (Slide 11)

9. Cardinality

Cardinality defines the number of instances of an entity involved in a relationship.

Types:

- 1:1 (One-to-One)
- 1:M (One-to-Many)
- M:N (Many-to-Many)

10. Practice Question

Generate an ER Diagram based on the scenario below:

- Patient details will be provided with values: ID [PK], Name, Age, and Visit Date.
- Test Reports will be provided with values: Name [PK], Date, and Result.
- A Patient will visit a Doctor, where Doctor has values: ID [PK], Name, and Specialization.

MySQL Learning Scenario: The "Kitab

Ghar" Bookstore

This document contains a complete MySQL learning scenario based on an Indian online bookstore context. It includes 6 tables, DDL/DML scripts, and 3 levels of practice questions.

BOOKSTORE MANAGEMENT SYSTEM – ER DIAGRAM

